



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«ДОНСКОЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(ДГТУ)

ОТЧЁТ

по лабораторной работе № 5

«Запуск открытой модели text-to-image»

по дисциплине

«Искусственный интеллект в креативных технологиях»

Вариант 2

бриф 1 «афиша фестиваля робототехники» × условие 2 «seed и seed + 1»

Выполнил: Яковенко Максим Михайлович

Группа: Т26ИИКТ-МИК11

1. Титульные данные и вариант

Номер варианта раскладывается по правилу раздела 7 указаний $N = (\text{номер брифа} - 1) \cdot 6 + \text{номер условия}$. Для $N = 2$ это бриф 1 и условие 2; seed варианта равен $1000 + N = 1002$. В ноутбуке то же правило применяет функция преподавателя `variant_summary(pipe, 2)`, а ячейка проверяет, что она выбрала бриф 1, условие `seed_plus_1` и seed 1002.

Таблица 1 — Параметры варианта

Параметр	Значение
Исполнитель	Яковенко Максим Михайлович, группа Т26ИИКТ-МИК11
Вариант	2
Бриф 1	афиша фестиваля робототехники: силуэт механической руки, синий и янтарный
Условие 2	seed и seed + 1 (<code>seed_plus_1</code>): A — seed 1002, B — seed 1003
Фиксировано	модель и ревизия, запрос, 512×512 , 1 шаг, <code>guidance_scale = 0,0</code> , генератор на CPU, float16, метрики и допуск 40 дБ
Требуемый артефакт	блокнот с выводом; <code>result.png</code> и <code>manifest.json</code> для условий A и B; таблица SHA-256, совпадения пикселей, максимального отличия, PSNR, вердикта; паспорт эксперимента

2. Цель, бриф, условие, гипотеза и допуск

2.1. Цель

Запустить открытую модель `text-to-image` по закреплённой ревизии, сохранить изображение вместе с манифестом условий запуска и измеримо, а не по впечатлению, ответить на вопрос, что меняется при смене seed на единицу: воспроизводится ли результат и что именно фиксирует seed.

2.2. Бриф и условие сравнения

Бриф 1 задаёт концепт афиши фестиваля робототехники. Условие 2 меняет ровно один параметр — seed: запуск A выполняется с seed 1002, запуск B — с seed 1003, всё остальное (модель, ревизия, запрос, число шагов, `guidance`, размер, устройство генератора, тип вычислений, среда и сеанс) одинаково. Это подтверждают манифесты `condition_A/manifest.json` и `condition_B/manifest.json`: они различаются только полями `seed`, `sha256` и `condition`. По условию варианта дополнительно оценивается глазами, узнаётся ли на обоих изображениях силуэт механической руки и остаётся ли он главным объектом кадра.

2.3. Гипотеза и допуск, записанные до запуска

Гипотеза и допуск записаны в журнал `reports/journal.md` и зафиксированы в репозитории коммитом `be8fcd9` до первой генерации по варианту; после запусков они не менялись. В ноутбуке они повторены в разделе «Шаг 2».

Допуск (раздел 2.3 указаний): пара «идентична» (`identical`) при побитовом совпадении пикселей, «практически совпадает» (`close`) при $\text{PSNR} \geq 40$ дБ, «различна» (`different`) при $\text{PSNR} < 40$ дБ. Порог 40 дБ условный: он отделяет численный шум вычислений (порядок

40–50 дБ при смене пакета или точности при общем шуме) от смены содержания кадра (порядок 10 дБ).

Механизм, из которого выведена гипотеза. Seed задаёт только начальный шум латентного пространства — тензор $1 \times 4 \times 64 \times 64$. Генератор с seed 1003 выдаёт последовательность, статистически независимую от seed 1002, то есть это другой вход модели. SD Turbo делает один шаг шумоподавления, поэтому изображение почти целиком определяется этим шумом. Ориентир по величине — намеренная ошибка демо указаний, где второй запуск тоже получает другой шум: PSNR 11,14 дБ.

Таблица 2 — Ожидания до запуска

Что проверяется	Ожидание	Обоснование
SHA-256 А и В	различны	другой шум даёт другие пиксели
совпадение пикселей	нет; максимум различия близок к 255	меняется содержание кадра, а не отдельные пиксели
PSNR A/B	порядка 8–15 дБ, вердикт different	ориентир — 11,14 дБ у намеренной ошибки демо
содержание	рука узнаётся и главная на обоих, палитра синий/янтарный сохранена	запрос один и тот же
повтор с тем же seed	побитовое совпадение	свежий генератор, та же среда
общий генератор на два вызова	второе изображение другое, PSNR < 40 дБ	состояние генератора сдвигается
новый сеанс	SHA-256 совпадёт с первым сеансом	те же версии и та же модель GPU

Направленность гипотезы: предсказаны не только знак результата (различие), но и его масштаб (8–15 дБ, далеко ниже порога), и то, что различие не является нестабильностью модели.

3. Данные, лицензия модели и ограничения

3.1. Данные

Входные данные — только текст запроса: бриф 1 из списка BRIEFS учебной библиотеки, без изменений:

«editorial poster concept for a robotics festival, glowing mechanical arm silhouette, deep blue and amber palette, clean composition, no text, no logo, no people»

Остальные брифы библиотеки (обложка подкаста, постер выставки, визуал лектория, обложка релиза) в варианте не используются. В запросе нет реальных людей, логотипов, названий производителей и закрытых материалов; изображение — концепт, а не иллюстрация реального устройства. Посторонние изображения в работе не используются: все рисунки отчёта получены запуском модели в ноутбуке.

3.2. Лицензия модели

Файл `LICENSE.md` в репозитории `stabilityai/sd-turbo` на закреплённой ревизии содержит Stability AI Community License Agreement от 5 июля 2024 г. (раздел 2.5 указаний). Из неё для данной работы следует:

1. некоммерческое, в том числе учебное, использование разрешено; работа выполнена в учебных некоммерческих целях;
2. коммерческое использование разрешено организациям с годовым доходом менее 1 млн долларов США и требует регистрации у правообладателя — в работе не применяется;
3. при распространении модели или результатов сохраняются уведомление о лицензии и указание «Powered by Stability AI»; в отчёте и README указаны лицензия и это уведомление;
4. действует политика допустимого использования (Acceptable Use Policy); запрос ей соответствует — нет людей, логотипов и вводящего в заблуждение содержания.

Ссылка на лицензию: <https://huggingface.co/stabilityai/sd-turbo/blob/b261bac6fd2cf515557d5d0707481eafa0485ec2/LICENSE.md>. «Открытая модель» здесь означает доступность весов и документации, а не соответствие лицензии критериям открытого ПО.

3.3. Ограничения

1. **Одна пара seed и один бриф.** Вердикт `different` для `seed + 1` ожидаем всегда, но число 10,63 дБ для другой пары будет другим.
2. **Одна среда.** Побитовая воспроизводимость проверена только в Colab на Tesla T4 с `torch 2.11.0+cu130` и `float16`.
3. **Свойства модели** (по карточке): один шаг, `guidance 0,0`, рекомендуемое разрешение 512×512 ; читаемый текст модель не выводит, люди и лица получаются неверно — поэтому бриф их исключает.
4. **PSNR** — численное отличие, а не оценка качества; визуальная оценка субъективна и сделана одним наблюдателем.

4. Среда выполнения

Значения взяты из вывода ячеек «Шаг 1» ноутбука и из `manifest.json`. Вместо команд `nvidia-smi` и `pip list` раздела 4 указаний те же сведения получены средствами Python (`torch.cuda`, `importlib.metadata`), чтобы они сразу попадали в манифест.

Таблица 3 — Среда выполнения

Параметр	Значение
Площадка	Google Colab, ускоритель «Графический процессор T4»
GPU	Tesla T4, 14,6 ГиБ; CUDA в сборке <code>torch</code> — 13.0
CPU и память	2 ядра, ОЗУ 12,7 ГиБ
Платформа	Linux-6.6.122+-x86_64-with-glibc2.39
Python	3.13.15

Параметр	Значение
Пакеты	torch 2.11.0+cu130 (предустановлен); diffusers 0.40.0; transformers 5.16.1 (доустановлен первой ячейкой); Pillow 11.3.0; numpy 2.1.3
Модель	stabilityai/sd-turbo, ревизия b261bac6fd2cf515557d5d0707481eafa0485ec2, файлы fp16, safetensors, без авторизации
Библиотека	lab05_lib.py из приложения А указаний: 6447 байт, SHA-256 4eb17650... — совпал, исполнялась без изменений
Устройство и точность	конвейер на cuda, float16; генератор шума на CPU

По сравнению с проверенной конфигурацией указаний (Colab, Tesla T4, Python 3.13.15, torch 2.11.0+cu128, diffusers 0.40.0, transformers 5.16.1, Pillow 11.3.0) отличается только сборка torch: +cu130 вместо +cu128, то есть CUDA 13.0 вместо 12.8. Модель GPU, версии Python-пакетов и платформа те же. Последствия этого отличия разобраны в разделе 7.3.

Запасной путь без GPU (замена "cuda" → "cpu" и float16 → float32 в памяти, без изменения файла библиотеки) в ноутбуке предусмотрен, но не сработал: CUDA была доступна, в манифест записано lib_adaptation: «нет: CUDA доступна, библиотека используется без изменений».

5. Ход работы

Ноутбук построен по плану раздела 5 указаний: каждой ячейке кода предшествует пояснение. Ноутбук выполнен дважды — в первом сеансе и в новом сеансе после удаления и повторного создания среды выполнения; в файле сохранены выходы второго сеанса, результаты первого — в истории репозитория.

Таблица 4 — Соответствие шагов плана ячейкам ноутбука

Шаг	Ячейки ноутбука	Наблюдаемый результат
1. Среда	«Шаг 1»: пакеты, среда, %%writefile lab05_lib.py, сверка SHA-256; «Запасной путь без GPU»	Tesla T4, версии; SHA-256 библиотеки совпал
2. Гипотеза и допуск	«Шаг 2» (повтор записи из журнала, коммит be8fcd9)	записаны до запуска
3. Модель и прогрев	«Шаг 3»	загрузка 65,8 с, прогрев 3,8 с
4. Базовый запуск	«Шаг 4», «Проверка в новом сеансе»	403134b8..., повтор и новый сеанс совпали
5. Сравнение А/В	«Шаг 5», «Изображения пары А и В»	PSNR 10,63 дБ, different
6. Причина различия	«Шаг 6» — отдельный эксперимент с latents	шум воспроизводит А и В бит в бит
7. Намеренная ошибка	«Шаг 7»	PSNR 9,25 дБ, different
8. Паспорт и итог	«Шаг 8», демо преподавателя, «Вывод», «Ограничения», «Паспорт»	13 файлов в SHA256SUMS; демо 7daa342a...

5.1. Подготовка среды и закрепление версий

```
TORCH_V = installed_version("torch")
HAS_GPU = shutil.which("nvidia-smi") is not None or os.path.exists("/dev/nvidia0")
PINS = ({ "diffusers": "0.40.0", "transformers": "5.16.1" } if at_least(TORCH_V, (2, 6))
        else { "diffusers": "0.32.2", "transformers": "4.48.3" })
todo = [ f"{k}=={v}" for k, v in PINS.items() if installed_version(k) != v ]
if todo:
    subprocess.run([sys.executable, "-m", "pip", "install", "-q", *todo], check=True)
```

Что делает. Читает версию torch из метаданных пакета без импорта и ставит diffusers и transformers тех версий, на которых преподаватель проверял библиотеку; уже совпадающие пакеты не трогает. **Зачем.** Версии библиотек входят в условия запуска: от них зависят байты изображения. Импортированный torch нельзя заменить в работающем ядре, поэтому он проверяется по метаданным. В Colab был torch 2.11.0+cu130 и другая версия transformers — ячейка доустановила transformers 5.16.1.

```
CUDA = torch.cuda.is_available()
DEVICE_NAME = torch.cuda.get_device_name(0) if CUDA else cpu_name()
print("Память GPU, ГиБ:", round(torch.cuda.get_device_properties(0).total_memory / 1024 **
3, 1),
      "| CUDA в torch:", torch.version.cuda)
```

Что делает. Фиксирует устройство и версии: имя и память GPU, версию CUDA в сборке torch, версии пакетов, Python, платформу. **Зачем.** Это аналог nvidia-smi и pip list из раздела 4; значения попадают в манифест и паспорт, и именно они позволили найти причину расхождения демо с указаниями.

5.2. Библиотека преподавателя и проверка её целостности

```
%%writefile lab05_lib.py
# ... текст из приложения А указаний без правок ...

LIB_SHA = hashlib.sha256(lib_bytes).hexdigest()
LIB_OK = len(lib_bytes) == 6447 and LIB_SHA.startswith("4eb17650")
assert LIB_OK, "библиотека введена с ошибкой – сверить текст ячейки с приложением А"
```

Что делает. Записывает библиотеку в файл и сравнивает его размер и SHA-256 с указаниями. **Зачем.** Совпадение (6447 байт, 4eb176506585ffcd...) доказывает, что код модели и метрик побитово тот же, что у преподавателя, — иначе отличие результатов можно было бы списать на опечатку при вводе. Затем библиотека импортируется как есть (import lab05_lib as L), адаптация не применяется.

5.3. Загрузка модели и прогрев

```
# lab05_lib.load_pipe – функция преподавателя
pipe = AutoPipelineForText2Image.from_pretrained(
    MODEL_ID, revision=REVISION, torch_dtype=torch.float16, variant="fp16",
    use_safetensors=True)
return pipe.to("cuda", dtype)

# ячейка ноутбука
pipe = L.load_pipe()
DTYPE = str(pipe.unet.dtype).replace("torch.", "")
L.generate(pipe, L.BRIEFS[0], 1) # прогрев с посторонним seed 1
```

Что делает. Загружает конвейер по полной ревизии b261bac6... (файлы fp16) на GPU во float16; фактический тип вычислений пишется в манифест. **Зачем.** Ревизия закрепляет снимок весов. Прогрев убирает однократные накладные расходы из времени базового запуска и на результат не влияет: каждый запуск создаёт свой генератор.

5.4. Базовый запуск, манифест и повтор

```
# lab05_lib.generate – функция преподавателя: свой генератор на каждый вызов
gens = [torch.Generator(device=gen_device).manual_seed(seed + i) for i in range(batch)]
return pipe(prompt=[prompt] * batch, num_inference_steps=steps, guidance_scale=guidance,
            height=size, width=size, generator=gens).images

# ячейка ноутбука
PROMPT, SEED, STEPS, GUIDANCE, SIZE = L.BRIEFS[0], 1000 + N, 1, 0.0, 512
img = L.generate(pipe, PROMPT, SEED, STEPS, GUIDANCE, SIZE)[0]
data = L.png_bytes(img)
(out / "result.png").write_bytes(data)
manifest = dict(model_id=L.MODEL_ID, revision=L.REVISION, prompt=PROMPT, seed=SEED,
               steps=STEPS,
               # ... guidance, размер, устройство генератора, dtype, устройство, версии,
               время ...
               sha256=L.sha(data), lib_sha256=LIB_SHA, lib_adaptation=ADAPTATION)
again = L.generate(pipe, PROMPT, SEED, STEPS, GUIDANCE, SIZE)[0]
print("Повтор со свежим генератором, SHA-256 совпал:", L.sha(L.png_bytes(again)) ==
      manifest["sha256"])
```

Что делает. Генерирует изображение с seed варианта 1002, сохраняет PNG и рядом манифест со всеми условиями запуска и SHA-256 файла, затем повторяет запуск со свежим генератором и тем же seed. **Зачем.** Манифест превращает картинку в воспроизводимый результат: по нему видно, что именно было зафиксировано. Повтор проверяет базовое утверждение «тот же seed — тот же результат» до сравнения по условию; без него различие А и В нельзя было бы приписать seed.

```
FIRST_SESSION_SHA = "403134b8e34029d144d41b14a9de11273a7a6dc3defb79fc871aa181b48faf1d" #
первый сеанс
print("Новый сеанс: SHA-256 базового запуска совпал с первым сеансом:", manifest["sha256"]
      == FIRST_SESSION_SHA)
```

Что делает. Сравнивает SHA-256 текущего сеанса с записанным в первом сеансе. **Зачем.** После удаления среды выполнения Colab выдаёт новую виртуальную машину с пустой папкой, поэтому сравнить с файлом первого сеанса можно только по записанной сумме. Это проверка воспроизводимости между сеансами — требование продвинутого уровня рубрики.

5.5. Сравнение по условию варианта

```
# lab05_lib.make_pair, ветка условия 2
elif cond == "seed_plus_1":
    a = generate(pipe, prompt, seed)[0]; b = generate(pipe, prompt, seed + 1)[0]

# lab05_lib.psnr_db
mse = np.mean((to_arr(a) - to_arr(b)) ** 2)
return float("inf") if mse == 0 else 10 * np.log10(255.0 ** 2 / mse)

# ячейка ноутбука
S = L.variant_summary(pipe, N)
assert (S["ctx"], S["cond"], S["seed"]) == (1, "seed_plus_1", SEED)
```

Что делает. Функция преподавателя строит пару A (seed 1002) и B (seed 1003) в одном сеансе и считает SHA-256, совпадение пикселей, максимум различия, PSNR и вердикт по допуску 40 дБ; `assert` гарантирует, что выбраны бриф, условие и seed варианта. **Зачем.** Метрики считаются кодом преподавателя и сопоставимы с указаниями; результат сохраняется в `variant_02_summary.json`.

```
img_b = L.generate(pipe, PROMPT, SEED + 1)[0]
print("A совпадает с A из variant_summary:", L.sha(L.png_bytes(img_a)) == S["sha_a"])
print("B совпадает с B из variant_summary:", L.sha(L.png_bytes(img_b)) == S["sha_b"])
```

Что делает. Получает изображения пары заново и сверяет их SHA-256 с суммами из `variant_summary`, затем сохраняет `pair/`, `condition_A/` и `condition_B/` с манифестами. **Зачем.** `variant_summary` возвращает только числа; сверка доказывает, что на рисунках ровно та пара, по которой посчитаны метрики, а не похожие изображения.

5.6. Отдельный эксперимент о причине различия

```
shape = (1, pipe.unet.config.in_channels, SIZE // 8, SIZE // 8)
noise = {s: torch.randn(shape, generator=torch.Generator(device="cpu").manual_seed(s),
dtype=pipe.unet.dtype)
        for s in (SEED, SEED + 1)}
na, nb = (noise[s].float().flatten().numpy() for s in (SEED, SEED + 1))
print(np.array_equal(na, nb), np.corrcoef(na, nb)[0, 1])
for s, ref, tag in ((SEED, sha_a, "A"), (SEED + 1, sha_b, "B")):
    x = pipe(prompt=PROMPT, num_inference_steps=STEPS, guidance_scale=GUIDANCE,
height=SIZE, width=SIZE,
        latents=noise[s].to(pipe.device)).images[0]
    print(tag, L.sha(L.png_bytes(x)) == ref)
```

Что делает. Строит начальный шум для seed 1002 и 1003 тем же способом, что и конвейер (генератор на CPU, тип вычислений конвейера), сравнивает тензоры и передаёт готовый шум в конвейер через аргумент `latents` — без генератора. **Зачем.** Шаг 6 плана (`psnr_fixed_noise`) относится к условию 4 и для варианта 2 возвращает `None`. Чтобы не принимать причину различия на веру, она проверена отдельно: если один только шум воспроизводит A и B бит в бит, то всё различие пары объясняется шумом, а не состоянием конвейера, прогревом или порядком вызовов.

5.7. Намеренная ошибка: общий генератор

```
g = torch.Generator(device="cpu").manual_seed(SEED)
e1 = pipe(prompt=PROMPT, num_inference_steps=STEPS, guidance_scale=GUIDANCE,
generator=g).images[0]
e2 = pipe(prompt=PROMPT, num_inference_steps=STEPS, guidance_scale=GUIDANCE,
generator=g).images[0]
psnr_e = L.psnr_db(e1, e2)
```

Что делает. Повторяет ошибку из демо указаний на своём брифе и seed: один объект `Generator` передаётся в два вызова подряд. **Зачем.** Показать на своих данных типичную ошибку воспроизводимости и проверить её диагноз. Исправление — создавать генератор внутри каждого запуска (`torch.Generator(device="cpu").manual_seed(seed)`), как это делает `L.generate`.

5.8. Контрольные суммы и демонстрационный скрипт

```
for p in sorted(ART.rglob("*")):
    if p.is_file() and p.name != "SHA256SUMS":
        rel = p.relative_to(ART).as_posix()
        h = hashlib.sha256(p.read_bytes()).hexdigest()
        lines.append(f"{h} {rel}")
(ART / "SHA256SUMS").write_text("\n".join(lines) + "\n", encoding="utf-8")
```

Что делает. Записывает SHA-256 всех файлов в `artifacts/SHA256SUMS` в формате `sha256sum`. **Зачем.** По этому списку можно проверить, что файлы, перенесённые из Colab в репозиторий, не изменились; проверка `shasum -a 256 -c SHA256SUMS` в репозитории даёт ОК для всех 13 файлов.

```
os.chdir(DEMO) # отдельная папка, чтобы не перезаписать результаты варианта
runpy.run_path(str(WORK / "lab05_demo.py"), run_name="__main__")
ref = "ac45a7ba8c514b22c1b4cbe0e5c3cfddec62ee3d076b633aeeeadbca43f8a04e9" # раздел 6.3
указаний, Colab T4
print("SHA-256 демонстрационного запуска совпал с указаниями:", got == ref)
```

Что делает. Запускает демонстрационный скрипт преподавателя без изменений (текст в ячейке побайтно совпадает с выданным файлом: 2677 байт, SHA-256 53b3b13f...) в отдельной папке и сравнивает SHA-256 его изображения с указаниями. **Зачем.** Это контроль среды: если демо воспроизводится до бита, среда совпадает с проверенной; если нет — граница воспроизводимости проходит через какое-то отличие среды, и его нужно найти.

6. Результаты

Все числа — из вывода ячеек ноутбука (второй сеанс) и файлов `artifacts/`; значения первого сеанса — из истории репозитория.

6.1. Манифест базового запуска

Таблица 5 — Манифест `artifacts/run_001/manifest.json`

Поле	Значение
model_id, revision	stabilityai/sd-turbo, b261bac6fd2cf515557d5d0707481eafa0485ec2

Поле	Значение
prompt	бриф 1 (раздел 3.1)
seed, steps, guidance_scale, size	1002, 1, 0,0, 512
generator_device, dtype	cpu, float16
device, platform, python	Tesla T4, Linux-6.6.122+-x86_64-with-glibc2.39, 3.13.15
packages	torch 2.11.0+cu130, diffusers 0.40.0, transformers 5.16.1, pillow 11.3.0
load_seconds, generation_seconds	65,8 (со скачиванием весов), 0,33
image_size, image_mode	[512, 512], RGB
sha256	403134b8e34029d144d41b14a9de11273a7a6dc3defb79fc871aa181b48faf1d
lib_sha256 = lib_executed_sha256	4eb176506585ffcd1702605a767309fb451808229c3622f80ce882b7fa2ce1a
lib_adaptation	нет: CUDA доступна, библиотека используется без изменений

Манифест содержит все поля чек-листа указаний: модель, ревизию, seed, шаги, guidance, размер, устройство генератора, тип вычислений, версии и SHA-256. Манифесты condition_A и condition_B повторяют его, отличаясь только seed, sha256 и меткой условия; поле generation_seconds в них пустое, так как время измерялось для пары в целом (0,64 с).

6.2. Сравнение А и В

Таблица 6 — Сравнение по условию варианта: variant_summary(pipe, 2)

Величина	А	В
seed	1002	1003
SHA-256 PNG	403134b8e34029d144d41b14a9de11273a7a6dc3defb79fc871aa181b48faf1d	098ebd438eb3befedd979cf5edc0a71ff403f3ae22a664e1644d4309f93eca08
файл	condition_A/result.png, pair/A_seed1002.png	condition_B/result.png, pair/B_seed1003.png

Таблица 7 — Метрики пары А/В

Метрика	Значение	Комментарий
совпадение пикселей	нет	
максимум модуля различия	254	из 255 возможных
MSE	≈ 5627	получено из PSNR: $255^2 / 10^{1,0628}$
PSNR	10,63 дБ	внутри прогноза 8–15 дБ
вердикт (допуск 40 дБ)	different	совпал с гипотезой
время пары	0,64 с	в сравнение не входит
psnr_fixed_noise	None	шаг 6 плана — только для условия 4

PSNR пересчитан по сохранённым файлам `pair/A_seed1002.png` и `pair/B_seed1003.png` вне Colab — совпал до последнего знака (10,628205374321828 дБ), максимум различия тот же, 254.

6.3. Изображения пары и визуальная оценка

A: seed 1002



B: seed 1003 PSNR 10.63 dB, different



Рисунок 1 — изображения A (seed 1002, слева) и B (seed 1003, справа), файл `artifacts/pair/comparison.png`; подпись над B сформирована ячейкой ноутбука

A (seed 1002). В центре — многозвенная конструкция из янтарно-оранжевых манипуляторов с синими узлами на платформе; верхний манипулятор с захватом поднят влево вверх. За ней — светлый круг-ореол с янтарной каймой, фон тёмно-синий. Механическая рука узнаётся сразу, конструкция занимает около двух третей кадра и однозначно главный объект.

B (seed 1003). В центре — механическая фигура с тёмным корпусом и синим «глазом», два янтарных манипулятора подняты в стороны; за ней янтарный диск, фон — тёмно-синее звёздное небо, внизу слева — янтарная шестерня. Механические руки узнаются, объект стоит в центре и остаётся главным.

Общее и отличия. Палитра синий/янтарный сохранилась на обоих; текста, логотипов и людей нет. Смена seed изменила позу и число манипуляторов, фон и распределение света, но не смысл запроса. Отклонение от брифа есть на обоих: вместо силуэта (плоской тёмной фигуры) модель нарисовала объёмную детализированную конструкцию, местами похожую на робота целиком, а по краям — мелкие посторонние детали (в A — эскизы механизмов в углах, в B — кран справа). Раз это проявляется при обоих seed, это свойство модели и запроса при одном шаге, а не следствие условия варианта.

Итог визуальной оценки: требование варианта выполнено — на обоих изображениях механическая рука узнаётся и остаётся главным объектом кадра.

6.4. Намеренная ошибка

Таблица 8 — Общий генератор на два вызова, seed 1002 (error_shared_generator/result.json)

Величина	Значение
SHA-256 первого запуска	403134b8e34029d144d41b14a9de11273a7a6dc3defb79fc871aa181b48faf1d — совпал с эталоном
SHA-256 второго запуска	0e1a569aa69b4ab77ccb61c79e75188089f77be3cfe33679d6c09540c5b49361 — не совпал
максимум различия	255
PSNR между запусками	9,25 дБ, вердикт different

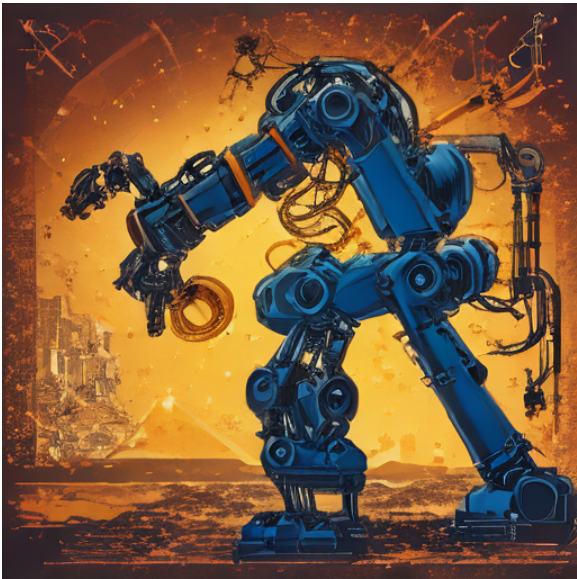


Рисунок 2 — намеренная ошибка: первый (слева, совпадает с A) и второй (справа) вызов с общим генератором, файлы artifacts/error_shared_generator/run1.png и run2.png

Второе изображение — другая композиция с обратной палитрой: синяя конструкция на янтарном фоне.

6.5. Проверка в новом сеансе

Таблица 9 — Первый и второй сеанс (среда выполнения удалена и создана заново)

Величина	Сеанс 1	Сеанс 2	Совпало
SHA-256 A (run_001)	403134b8...	403134b8...	да
SHA-256 B	098ebd43...	098ebd43...	да
PSNR пары	10,628205374321828	10,628205374321828	да
SHA-256 второго вызова с общим генератором	0e1a569a...	0e1a569a...	да
SHA-256 демо (seed 20260919)	7daa342a...	7daa342a...	да
эксперимент с latents	не выполнялся	A и B бит в бит	—
загрузка модели, с	38,1	65,8	не сравнивается
генерация, с	0,39	0,33	не сравнивается

В git-истории файлы PNG второго сеанса побайтно совпадают с первым; в манифестах изменились только поля времени, поэтому изменились и их SHA-256 в SHA256SUMS. Время загрузки и генерации зависит от сети и состояния GPU и в сравнение не входит.

6.6. Демонстрационный вариант преподавателя

Таблица 10 — Демо: получено в обоих сеансах и указано в разделе 6.3 указаний

Величина	Получено	В указаниях	Совпало
GPU	Tesla T4	Tesla T4	да
Python, diffusers, transformers, Pillow	3.13.15, 0.40.0, 5.16.1, 11.3.0	3.13.15, 0.40.0, 5.16.1, 11.3.0	да
torch	2.11.0+cu130	2.11.0+cu128	нет
код демо и библиотеки	53b3b13f..., 4eb17650...	53b3b13f..., 4eb17650...	да
SHA-256 изображения	7daa342ac2719f61...	ac45a7ba8c514b22...	нет
повтор в сеансе	совпал	совпал	да
общий генератор: 1-й / 2-й с эталоном	да / нет	да / нет	да
PSNR ошибки	11,15 дБ	11,14 дБ	нет, разница 0,01 дБ

7. Анализ

7.1. Причина различия А и В

Seed не задаёт изображение напрямую: он инициализирует генератор псевдослучайных чисел, из которого строится начальный латентный шум $1 \times 4 \times 64 \times 64$. Отдельный эксперимент (раздел 5.6) показал три вещи. Во-первых, шум seed 1002 и 1003 различен, а коэффициент корреляции между ними $-0,0018$ — соседние seed дают не «похожий», а независимый шум. Во-вторых, готовый шум seed 1003, поданный через latents без генератора, воспроизвёл В бит в бит. В-третьих, то же для seed 1002 и А. Значит, при фиксированных модели, запросе и параметрах изображение — детерминированная функция начального шума, и всё различие А и В (PSNR 10,63 дБ, максимум 254) вызвано сменой шума. Это смена входа модели, а не нестабильность.

Различие так велико, потому что SD Turbo делает один шаг: композицию определяет начальный шум, а запрос — только смысл (объект, палитру, стиль). Это видно на рисунке 1.

7.2. Ошибка и её исправление

Симптом: один объект Generator на два вызова — первое изображение совпало с эталоном (403134b8...), второе нет, PSNR 9,25 дБ. **Диагностика:** генератор хранит состояние; первый вызов расходует его на 16 384 значения шума $1 \times 4 \times 64 \times 64$ и сдвигает состояние, поэтому второй вызов получает следующие числа — фактически шум другого «seed». Величина 9,25 дБ того же порядка, что и смена seed (10,63 дБ): это смена содержания, а не численный шум. **Исправление:** создавать генератор внутри каждого запуска, как L.generate; повтор на шаге 4 со свежим генератором дал тот же SHA-256, то есть исправление работает.

7.3. Проверка в новом сеансе и границы воспроизводимости

В пределах среды воспроизводимость побитовая: повтор в сеансе и запуск в новом сеансе на новой виртуальной машине (веса скачаны заново, библиотека записана заново) дали те же SHA-256 для А, В, ошибочного запуска и демо, а PSNR пары совпал до последнего знака.

Граница воспроизводимости видна на демо: тот же код, seed 20260919 и та же модель GPU дали SHA-256 7daa342a... вместо ac45a7ba... из указаний. Из записанных параметров среды отличается только сборка torch: +cu130 (CUDA 13.0) против +cu128 (CUDA 12.8). Шум строится на CPU и от CUDA не зависит, но сеть шумоподавления и декодер считаются на GPU во float16 ядрами библиотек CUDA; другая версия CUDA означает другие ядра и другой порядок округлений, отсюда другие байты. Масштаб различия косвенно оценивается по PSNR ошибки: 11,15 дБ против 11,14 дБ — изображения демо разошлись на уровне численного шума, а не содержания. Напрямую PSNR с изображением указаний не измерен: само изображение в материалах не выдано. Версия драйвера NVIDIA не записана ни у меня, ни в указаниях, поэтому её вклад полностью исключить нельзя.

Это согласуется с документацией PyTorch и Diffusers (раздел 2.2 указаний): между сборками и платформами воспроизводимость гарантируется лишь «в пределах допуска». Утверждение «результат воспроизводим» относится к конфигурации Colab / Tesla T4 / torch 2.11.0+cu130 / float16.

7.4. Сопоставление с гипотезой

Таблица 11 — Гипотеза (коммит be8fcd9) и результат

Предсказание	Получено	Итог
вердикт different	different	сбылось
SHA-256 различны, пиксели не совпадают	да	сбылось
максимум различия близок к 255	254	сбылось
PSNR порядка 8–15 дБ	10,63 дБ	сбылось
рука узнаётся и главная на обоих, палитра сохранена	да; вместо силуэта — объёмная конструкция	сбылось с оговоркой
повтор с тем же seed — побитово	совпал	сбылось
общий генератор: второе другое, PSNR < 40 дБ	9,25 дБ	сбылось
новый сеанс — SHA-256 совпадёт	совпал	сбылось

7.5. Альтернативные объяснения

Различие А и В можно было бы объяснить не только сменой seed. Каждое объяснение проверено данными:

1. **Недетерминированность вычислений на GPU** (выбор алгоритмов, атомарные операции). Исключена: повтор с тем же seed и новый сеанс дали побитовое совпадение — случайность вычислений этого бы не позволила.
2. **Скрытое состояние конвейера** (прогрев, кэш, предыдущие вызовы). Исключено: изображение А получено пять раз после разного числа предыдущих вызовов (базовый

запуск, повтор, внутри `variant_summary`, первый вызов с общим генератором, эксперимент с `latents`) — всегда 403134b8....

3. **Различие параметров запуска.** Исключено: манифесты А и В различаются только `seed`, `sha256` и меткой условия.
4. **Артефакт метрики или формата.** Исключён: PNG сохраняет пиксели без потерь, PSNR пересчитан по файлам вне Colab и совпал.
5. **«Соседние seed дают похожие изображения».** Опровергнуто: корреляция шума $-0,0018$, PSNR 10,63 дБ того же порядка, что и у ошибки с общим генератором (9,25 дБ) и демо (11,15 дБ) — трёх пар с независимым шумом.
6. **Расхождение демо из-за кода или версий пакетов.** Исключено: код демо и библиотеки побайтно совпадает с выданным, версии Python-пакетов и модель GPU те же; остаётся сборка CUDA (и не записанный драйвер).

7.6. Угрозы валидности

1. **Внешняя валидность: одна пара seed, один бриф.** Число 10,63 дБ — оценка для одной пары; для других пар `seed` и брифов оно будет другим (в работе три независимые пары дали 9,25–11,15 дБ). Вердикт `different` от этого не зависит.
2. **Субъективность визуальной оценки.** Оценка «рука узнаётся и главная» сделана одним наблюдателем без шкалы; для строгости нужны несколько оценщиков и заранее заданные критерии.
3. **PSNR не измеряет восприятие.** 10,63 дБ говорит, что кадры численно разные, но не говорит, насколько они различаются для зрителя и насколько каждый соответствует брифу.
4. **Неполнота манифеста.** По образцу указаний записаны `torch`, `diffusers`, `transformers`, `Pillow`; версии драйвера NVIDIA, `accelerate` и `safetensors` в манифест не попали — это сужает возможность объяснить расхождения на другой машине.
5. **Незакреплённое окружение Colab.** `torch` предустановлен и может обновиться; через месяц тот же ноутбук может получить другую сборку CUDA и другие SHA-256 (как это произошло с демо).
6. **Порядок работы.** Эксперимент с `latents` добавлен после первого сеанса и выполнен во втором. Он проверяет механизм, а не подгоняет допуск: гипотеза и допуск после запуска не менялись.

8. Вывод

Смена `seed` на единицу при всех прочих фиксированных условиях даёт другое изображение — вердикт **different**, как и предсказывала гипотеза, записанная до запуска:

1. SHA-256 А и В различны, пиксели не совпадают, максимум различия 254, PSNR 10,63 дБ — внутри прогноза 8–15 дБ и далеко ниже допуска 40 дБ;
2. `seed` фиксирует только начальный латентный шум: готовый шум без генератора воспроизводит А и В бит в бит, а шум соседних `seed` не коррелирован ($-0,0018$);

- 3. seed не фиксирует модель и ревизию, запрос, шаги, guidance, размер, тип данных шума, устройство генератора, версии библиотек и сборку CUDA — последнее показал демо: та же T4 с torch+cu130 дала 7daa342a... вместо ac45a7ba...;
- 4. в среде Colab / Tesla T4 / torch 2.11.0+cu130 / float16 запуск воспроизводится побитово внутри сеанса и между сеансами;
- 5. намеренная ошибка с общим генератором даёт другое второе изображение (PSNR 9,25 дБ); исправление — свежий генератор на каждый запуск;
- 6. визуально на обоих изображениях механическая рука узнаётся и остаётся главным объектом, палитра сохранена; различие А и В — смена входа, а не нестабильность модели.

Границы вывода: одна пара seed, бриф 1, одна среда; на другой видеокарте или сборке CUDA SHA-256 будут другими, и сравнивать результаты там нужно по допуску, а не по контрольной сумме.

9. Блокнот, файлы, паспорт эксперимента и индивидуальный вклад

Репозиторий работы: <https://github.com/ykvnm/ai-in-creative-industries/tree/main/LR05>.
Блокнот с сохранёнными выходами — Яковенко_Максим_Михайлович_Вариант_2_ЛР_5.ipynb в этом каталоге.

Таблица 12 — Файлы работы

Путь в LR05/	Содержимое
Яковенко_Максим_Михайлович_Вариант_2_ЛР_5.ipynb	ноутбук Colab с выходами второго сеанса: код, пояснения, вывод, ограничения, паспорт
artifacts/run_001/	result.png и manifest.json базового запуска
artifacts/condition_A/, artifacts/condition_B/	result.png и manifest.json условий А и В
artifacts/pair/	А, В и сравнение бок о бок (рисунок 1)
artifacts/variant_02_summary.json	метрики пары из variant_summary
artifacts/error_shared_generator/	намеренная ошибка: run1.png, run2.png, result.json
artifacts/SHA256SUMS	SHA-256 всех 13 файлов артефактов
reports/journal.md	гипотеза и допуск до запуска, ход работы, сопоставление
assignment/	методические указания, библиотека, демо и журнал преподавателя

Таблица 13 — Паспорт эксперимента

Поле	Значение
Работа, вариант	ЛР №5, вариант 2: бриф 1 × условие 2 «seed и seed + 1»
Модель и ревизия	stabilityai/sd-turbo, b261bac6fd2cf515557d5d0707481eafa0485ec2, файлы fp16
Лицензия	Stability AI Community License Agreement (5 июля 2024 г.), Acceptable Use Policy; учебное некоммерческое использование; «Powered by Stability AI»
Запрос	BRIEFS[0] — бриф 1 (раздел 3.1)
seed	А — 1002, В — 1003; прогрев — 1; демо — 20260919
Шаги, guidance, размер	1; 0,0; 512 × 512

Поле	Значение
Генератор, тип вычислений	torch.Generator на CPU, свежий на каждый запуск; float16
Среда	Google Colab, Tesla T4 (14,6 ГиБ), CUDA 13.0; Linux-6.6.122+-x86_64-with-glibc2.39
Версии	Python 3.13.15, torch 2.11.0+cu130, diffusers 0.40.0, transformers 5.16.1, Pillow 11.3.0, numpy 2.1.3
Библиотека	lab05_lib.py без изменений, 6447 байт, SHA-256 4eb176506585ffcd...
SHA-256 A	403134b8e34029d144d41b14a9de11273a7a6dc3defb79fc871aa181b48faf1d
SHA-256 B	098ebd438eb3befedd979cf5edc0a71ff403f3ae22a664e1644d4309f93eca08
Допуск	identical — побитово; close — $\text{PSNR} \geq 40$ дБ; different — $\text{PSNR} < 40$ дБ; записан до запуска (коммит be8fcd9)
Результат	different, PSNR 10,63 дБ, максимум различия 254
Воспроизводимость	побитовая внутри сеанса и между сеансами в этой среде; демо на torch +cu130 $\neq$ указаниям (+cu128)
Границы	одна пара seed, один бриф, одна среда; на другой GPU или сборке CUDA — сравнение по допуску

Индивидуальный вклад. Работа выполнена мной полностью самостоятельно: формулировка гипотезы и допущения и их фиксация в репозитории до запуска; подготовка ноутбука (установка версий, сбор сведений о среде, проверка целостности библиотеки, запасной путь без GPU, манифесты, сохранение пары и контрольных сумм); отдельный эксперимент с latents для проверки причины различия; два прогона в Colab — в первом и в новом сеансе; запуск и сверка демо преподавателя; визуальная оценка, анализ и отчёт. Учебная библиотека lab05_lib.py и демонстрационный скрипт lab05_demo.py выданы преподавателем и использованы без изменений.

Приложение А. Ответы на контрольные вопросы

Вопросы приведены по разделу 9 методических указаний. Ответы опираются на данные настоящей работы.

1. Чем модель отличается от конвейера и от пользовательского приложения?

Модель — обученные веса нейросетей конкретной ревизии: текстовый кодировщик, сеть шумоподавления (UNet) и декодер изображения (VAE); в работе это `stabilityai/sd-turbo`, ревизия `b261bac6...`, файлы `fp16`. **Конвейер** — код, который соединяет эти части и задаёт процедуру: токенизация запроса, построение начального шума, число шагов планировщика, `guidance`, декодирование в пиксели. Здесь это `AutoPipelineForText2Image` из `diffusers 0.40.0` (при загрузке видно 5 компонентов). **Пользовательское приложение** — интерфейс поверх конвейера (веб-сервис, бот), который обычно скрывает `seed`, ревизию, шаги и версии; поэтому результат приложения нельзя воспроизвести, а в работе используется ноутбук, где все условия видны и записаны в манифест.

2. Почему имя модели без ревизии недостаточно для воспроизведения?

Имя `stabilityai/sd-turbo` указывает на текущее состояние репозитория (ветку `main`), которое владелец может изменить: заменить веса, конфигурацию планировщика или токенизатор. Тогда тот же код с тем же `seed` через какое-то время загрузит другие файлы и даст другое изображение. Ревизия — хеш коммита `b261bac6fd2cf515557d5d0707481eafa0485ec2` — неизменяемый снимок файлов. В работе она передаётся в `from_pretrained(revision=REVISION)` и записана в манифест; вместе с ней фиксируется и вариант файлов (`variant="fp16"`), потому что в одной ревизии лежат и `fp16`-, и полные веса.

3. Что фиксирует `seed` и чего он не фиксирует?

`Seed` фиксирует только последовательность псевдослучайных чисел генератора, то есть начальный латентный шум — при условии, что не меняются устройство генератора, тип данных и форма тензора. В работе это доказано прямо: готовый шум `seed 1002` и `1003` без генератора воспроизвёл А и В бит в бит, а смена `seed` на единицу дала независимый шум (корреляция $-0,0018$) и другой кадр (PSNR 10,63 дБ).

`Seed` не фиксирует: модель и ревизию, запрос, число шагов, `guidance`, размер (от него зависит форма шума), тип данных шума (`float16` и `float32` дают разные числа), устройство генератора (CPU и GPU дают разные числа), версии библиотек, сборку CUDA и модель GPU. Последнее показал демо: тот же `seed 20260919` на той же T4, но с `torch +cu130`, дал другой SHA-256, чем в указаниях (`+cu128`).

4. Почему Diffusers рекомендует генератор на CPU?

Генераторы CPU и GPU используют разные алгоритмы и при одном `seed` выдают разные числа. Генератор на CPU не зависит от модели видеокарты и версии CUDA, поэтому начальный шум одинаков на любой машине, даже без GPU; генератор на GPU привязывает шум к устройству. В работе генератор на CPU, поэтому расхождение демо с указаниями возникло не в шуме, а в вычислениях сети на GPU — причина сужена до одной части конвейера.

5. Почему один объект Generator, переданный в два запуска, даёт разные изображения? Что показала ваша проверка?

Generator хранит внутреннее состояние, которое сдвигается при каждом использовании. Первый вызов конвейера расходует генератор на $1 \cdot 4 \cdot 64 \cdot 64 = 16\,384$ значения шума; второй вызов продолжает с того места, где остановился первый, и получает другой шум — как если бы seed был другим. Проверка на моём брифе и seed 1002: первый вызов совпал с эталоном (403134b8...), второй — нет (0e1a569a...), PSNR 9,25 дБ, максимум различия 255, вердикт different. Величина того же порядка, что при смене seed (10,63 дБ). Исправление — новый генератор на каждый запуск, как в `L.generate`; с ним повтор совпал побитово.

6. Что доказывает совпадение SHA-256 двух файлов, а что не доказывает?

Доказывает: файлы совпадают побайтно (вероятность случайного совпадения SHA-256 пренебрежимо мала), значит, изображения в них идентичны. В работе одна и та же сумма 403134b8... у `run_001/result.png`, `condition_A/result.png`, `pair/A_seed1002.png` и `error_shared_generator/run1.png` — это один и тот же кадр, полученный разными путями, в том числе в другом сеансе.

Не доказывает: качества изображения и его соответствия брифу; того, что файл получен при записанных условиях (для этого нужен манифест); воспроизводимости на другой машине. Несовпадение SHA-256 не означает, что изображения разные по существу: другие метаданные или степень сжатия PNG меняют байты при тех же пикселях, а демо на другой сборке CUDA дало другой SHA-256 при, по косвенной оценке, численно близком изображении. Поэтому наряду с SHA-256 считаются пиксели и PSNR.

7. Найдите PSNR, если MSE двух изображений на шкале 0...255 равна 65,025

$\text{PSNR} = 10 \cdot \lg(255^2 / \text{MSE}) = 10 \cdot \lg(65\,025 / 65,025) = 10 \cdot \lg(1000) = 10 \cdot 3 = 30 \text{ дБ}$

$255^2 = 65\,025$, поэтому отношение равно ровно 1000, а десятичный логарифм — 3. Ответ: **30 дБ**. По допуску работы (40 дБ) такая пара получила бы вердикт different, хотя численно она гораздо ближе, чем пара моего варианта: у неё $\text{MSE} \approx 5627$ и PSNR 10,63 дБ. Для сравнения, повышение PSNR на 10 дБ соответствует уменьшению MSE в 10 раз: при 40 дБ MSE была бы $\approx 6,5$.

8. Почему при смене float16 на float32 и одном seed изображения различны, хотя веса те же? Как проверить причину?

Смена типа меняет не только точность вычислений, но и начальный шум: `torch.randn` с одним seed выдаёт для float16 и float32 разные числа (в журнале преподавателя максимум разности 5,30). Поэтому сравниваются фактически разные входы, и PSNR получается 7–16 дБ — как при смене seed. Чтобы разделить причины, нужно подать в оба режима одинаковый шум: функция `generate_fixed_noise` строит его всегда во float32 и приводит к типу вычислений. Тогда остаётся только различие точности, и PSNR поднимается до 37–51 дБ. Тот же приём — передать готовый шум через `latents` — я применил в своей работе для условия `seed + 1` и показал, что различие A и B целиком объясняется шумом.

9. Какие лицензионные условия применимы к вашему запуску и что нужно указать в отчёте?

Применима Stability AI Community License Agreement от 5 июля 2024 г. из LICENSE.md на ревизии b261bac6... и политика допустимого использования. Мой запуск — учебное некоммерческое использование, оно разрешено; коммерческое использование возможно для организаций с доходом менее 1 млн долларов США после регистрации, но в работе не применяется. При распространении изображений нужно сохранить уведомление о лицензии и указание «Powered by Stability AI». В отчёте указаны: название лицензии и ссылка на LICENSE.md на закреплённой ревизии, цель использования, соответствие запроса политике (нет реальных людей, логотипов, закрытых материалов) и то, что «открытая модель» не означает лицензию открытого ПО (раздел 3.2).

10. Что нужно изменить в паспорте, если запуск повторён на другой видеокарте?

Обновить поля среды: модель и память GPU, версию драйвера и CUDA, сборку torch, платформу и версии пакетов, если они изменились. Записать новые SHA-256 всех изображений как новые эталоны для этой среды и явно указать, что побитовое совпадение со старыми SHA-256 не ожидается и не проверяется. Изменить формулировку воспроизводимости: «побитово — в среде X; между средами — в пределах допуска PSNR ≥ 40 дБ» и добавить измеренный PSNR между изображениями старой и новой среды при одном seed. Повторить в новой среде проверки повтора и нового сеанса. Демо показывает, что это нужно даже без смены видеокарты: та же Tesla T4 с другой сборкой CUDA дала другой SHA-256.